

PIEDPIN

CISSP for BEGINNER

なぜCISSPを学ぶのか

- ✓ CISSP(Certified Information Systems Security Professional)とは、世界標準の**セキュリティ・プロフェッショナル認定**です。
- ✓ (ISC)²（国際情報システムセキュリティ認証コンソーシアム、International Information Systems Security Certification Consortium）が認定を行っています。
- ✓ 情報セキュリティ専門家が共通言語として利用している『(ISC)² CISSP CBK』を理解している者に与えられる資格です。
- ✓ 2004年6月に、米国規格協会（ANSI）より**ISO/IEC17024**の認証を受け、信頼度がより高くなりました。（https://japan.isc2.org/cissp_about.html）
- ✓ 2022年7月1日現在、日本のCISSPは**3,699**人です。

(ISC)² が展開する資格ソリューション



国際的に認められた情報セキュリティ・プロフェッショナル認定資格です。米国規格協会（ANSI）よりISO/IEC17024の認証を受けた。



ベンダーフリー・カントリーフリーの情報セキュリティの資格です。情報セキュリティに従事しないが、組織を理解し、適切なコミュニケーションを行う。



クラウドサービスを安全に利用するために必要な知識を体系化した資格です。



ソフトウェアの脆弱性に端を発したセキュリティインシデントの増加や、政府や重要インフラにおけるセキュアなソフトウェアニーズに対応する資格です。

(ISC)² CISSP CBK ドメイン



ドメイン 1
セキュリティとリスク
マネジメント



ドメイン 2
資産のセキュリティ



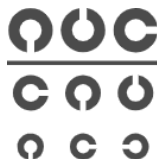
ドメイン 3
セキュリティアーキ
テクチャとエンジニ
アリング



ドメイン 4
通信とネットワー
クのセキュリティ



ドメイン 5
アイデンティティと
アクセスの管理



ドメイン 6
セキュリティの評
価とテスト



ドメイン 7
セキュリティの運用



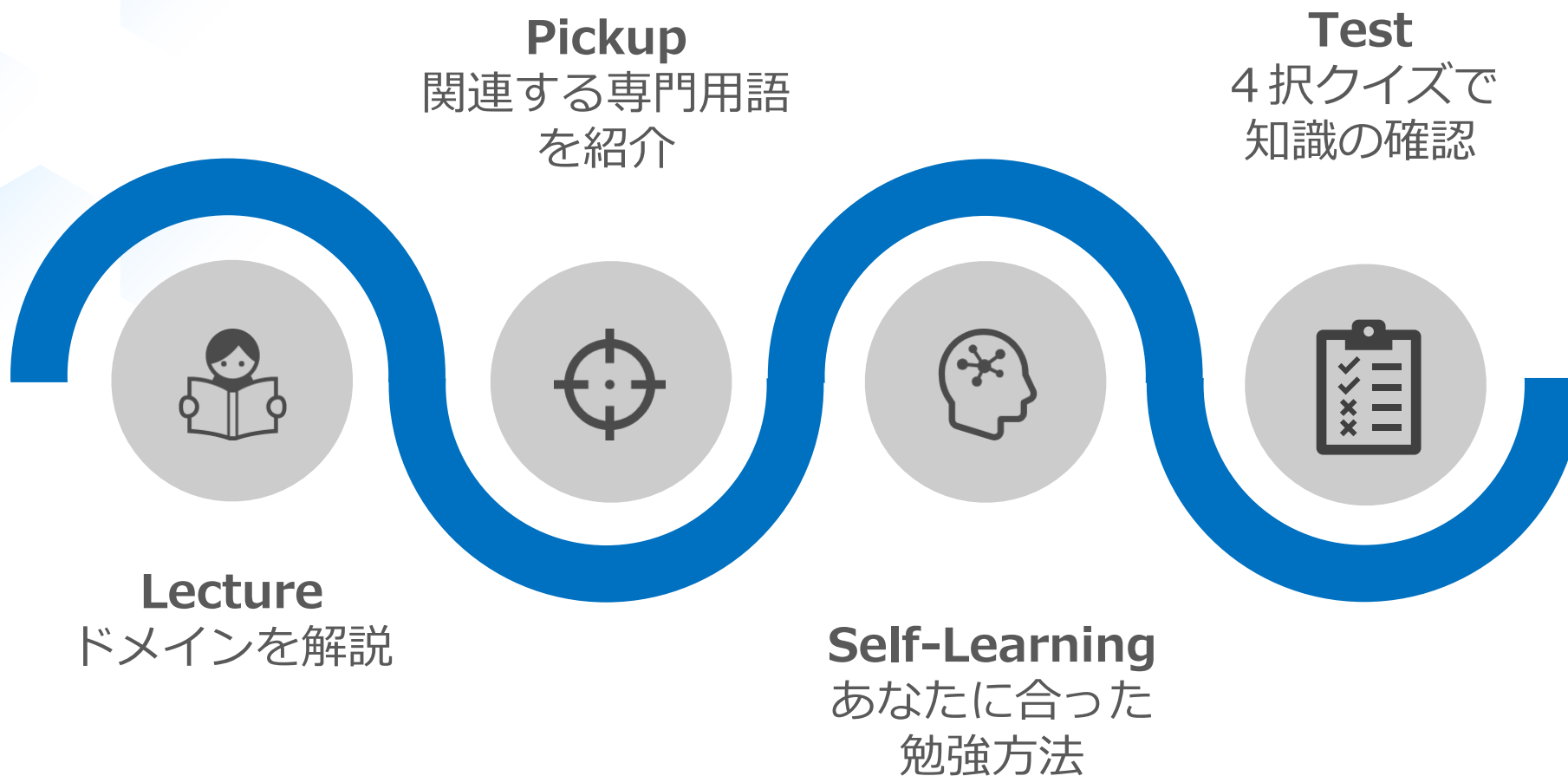
ドメイン 8
ソフトウェア開発
セキュリティ

あなたのゴールと本レッスンの内容

- ✓ 情報セキュリティ初心者のために設計されたCISSP講座
経験者からすると“当たり前”と感じる内容も含まれます。
- ✓ 本レッスンの対象は、CBKのドメイン 8 です。
他のドメインのレッスンは対象外となります。
- ✓ CISSP合格のためのアップグレードが必要です。
現代のセキュリティの関心、流行、考え方を織り込みます。



レッスンの流れ



ドメイン別試験比重



ドメイン 1
セキュリティと
リスクマネジメント

16%



ドメイン 2
資産のセキュリティ

10%



ドメイン 3
セキュリティアーキテク
チャとエンジニアリング

13%



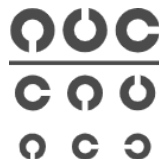
ドメイン 4
通信とネットワーク
のセキュリティ

13%



ドメイン 5
アイデンティティと
アクセスの管理

13%



ドメイン 6
セキュリティの評価
とテスト

12%



ドメイン 7
セキュリティの運用

13%



ドメイン 8
ソフトウェア開発
セキュリティ

10%



開発環境

開発環境

プログラミング言語

●プログラム 機械語

- 機械語とは、CPUによって直接実行されるソフトウェアです。
- 機械が読める言語とは、CPUが読む言語です。CPUによって理解されるのは0と1の世界であり、人間が理解できるものではありません。

ソースコード

- ソースコードとは、処理内容を人間が理解できるテキストで記載されたものです。
- コンピュータプログラミング言語としてコンピュータで実行する場合には、一旦機械語に翻訳されています。

バイトコード

- バイトコードを利用するプログラミング言語やその実行環境は、実行時に仮想マシンによってそのコンピュータのCPU固有のネイティブコードに変換して実行します。
- 特定のCPUの仕様に関わらず、環境には依存せず適応させることができます。

ソースコード

```
If (test == 1) {  
...  
}
```

コンパイル

バイトコード（中間コード）



実行
機械語に変換

```
00101101011101010  
10110101
```



開発環境

プログラミング言語

●プログラミング言語

4世代言語（4GL）

- 4世代言語は、プログラミングの労力を軽減し、ソフトウェア開発の時間を短縮するように設計された言語です。そのため、開発コストを抑えることができるというメリットがあります。

アセンブリ言語

- アセンブリ言語は、CPUの処理を直接記述する低水準言語であり、ADD/SUB/JMP などの命令を用いた値の操作が中心です。
- 機械語（0と1）を人間にも理解しやすい英字の略語で表現したのですが、基本的には値の交換や簡単な計算しか記述できず、Webシステムのような高度なアプリケーションを開発するのには適していません。
- アセンブラはアセンブリ言語を機械語に変換するツールであり、逆アセンブラは機械語をアセンブリ言語に変換するツールです。

世代	名前	例
第1世代	機械語	機械語
第2世代	アセンブリ言語	アセンブリ
第3世代	高級言語	Cobol、C、C ++、C #、Java、JavaScript
第4世代	超高級言語	ColdFusion、Progress 4GL、SQL、PHP、Perl
第5世代	自然言語	LISP、Prolog、人工知能

開発環境

ランタイム

●プログラムの実行 コンパイラ言語

- 高級言語を機械語に変換して実行可能ファイルとして保存します。1度コンパイルした実行ファイルはその後はコンパイルなく実行できます。
- 例：C、C++など

インタプリタ言語

- 一行ずつ実行されるたびに機械語に翻訳されます。
- 例：Javascript、Python、PHPなど

コンパイラ言語

一括翻訳

```
int main() {  
    print "Hello World"  
    print "Hello Coding"  
    return 0;  
}
```

コンパイル

```
01010101101011101  
01010101010101011  
10110101010101010
```

インタプリタ言語

一行ずつ翻訳

```
print "Hello World"  
print "Hello Coding"
```

```
print "Hello World"
```

```
01010101101101010
```

```
print "Hello Coding"
```

```
10110100101010110
```

開発環境

プログラミング

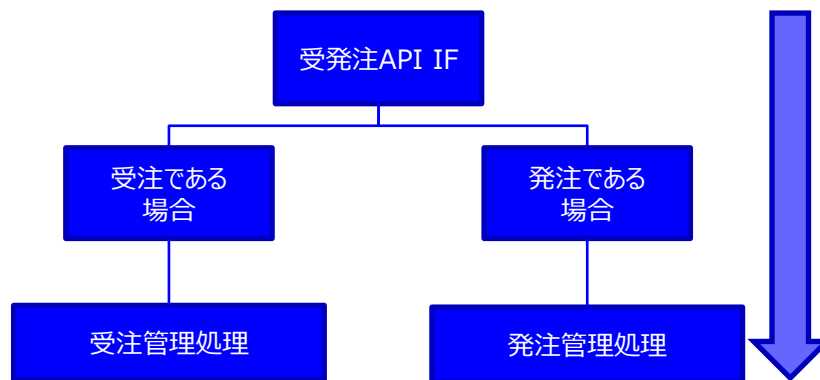
●プログラミング

トップダウンプログラミング

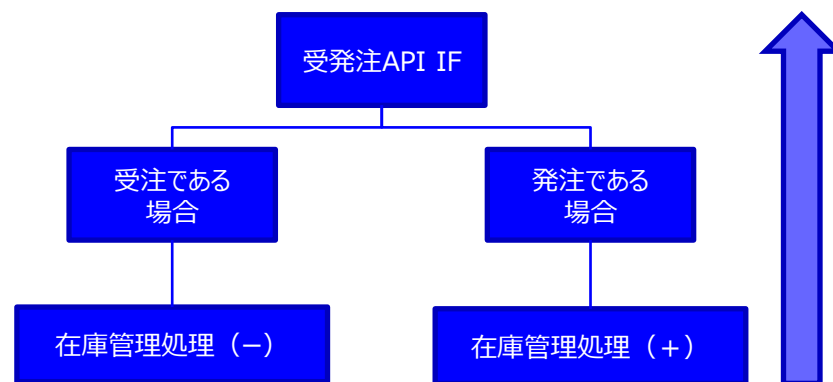
- 全体像から始めて、それを小さなセグメントに分割しながら開発する手法です。
- 全体の仕様が基本要素に細分化されるまで、各サブシステムの処理内容がさらに詳細にして改良されます。

ボトムアッププログラミング

- 小さなセグメントをつなぎ合わせてより複雑な大きなシステムを開発する手法です。
- システムの基本要素は処理内容が成立しています。次にそれらが組み合わされることにより、大きなサブシステムを形成します。
- 個々の処理としては完結しており、そのインプットをどのようにするのかでその処理上での意味を持たせます。



トップダウンではシステムの構成要素を開発していく



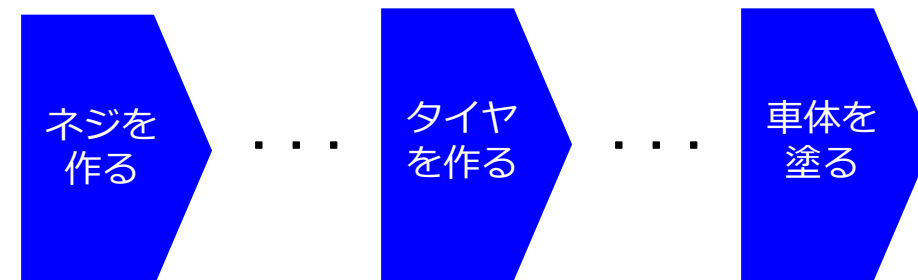
ボトムアップでは要素を組み合わせてシステムを開発していく

開発環境

プログラミング

● 手続き型言語 (Procedural Language)

- 手続き型言語 (Procedural Language) とは、プログラムを手順（手続き）の集合として記述し、順序に従って処理を実行するプログラミング言語のパラダイムです。
- 「何をするか (What)」ではなく、「どのように実行するか (How)」を明確に記述するのが特徴です。
- コードの流れが明確で、学習しやすいが、大規模なシステムには管理が難しくなることもある。



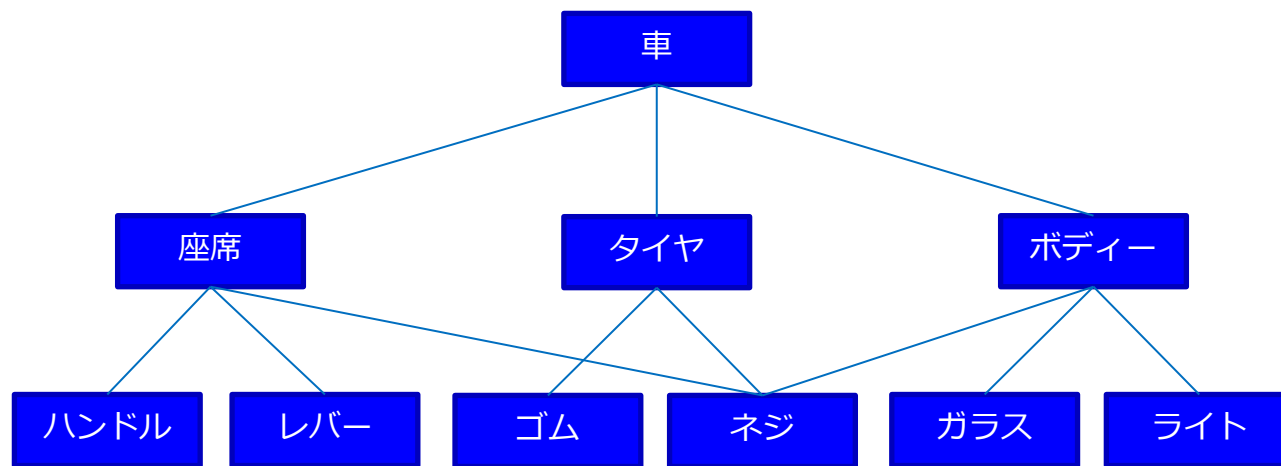
手続き型言語の一例（車を作る）

開発環境

プログラミング

●オブジェクト指向言語（OOP、Object-Oriented Language）

- オブジェクト指向言語（Object-Oriented Programming Language）とは、データ（属性）と処理（メソッド）をオブジェクトとしてまとめ、プログラムを構築するプログラミング言語です。
- 従来の手続き型言語（Procedural Language）が「手続き（処理の順番）」に焦点を当てるのに対し、オブジェクト指向言語は、「データ（オブジェクト）」を中心に設計されます。



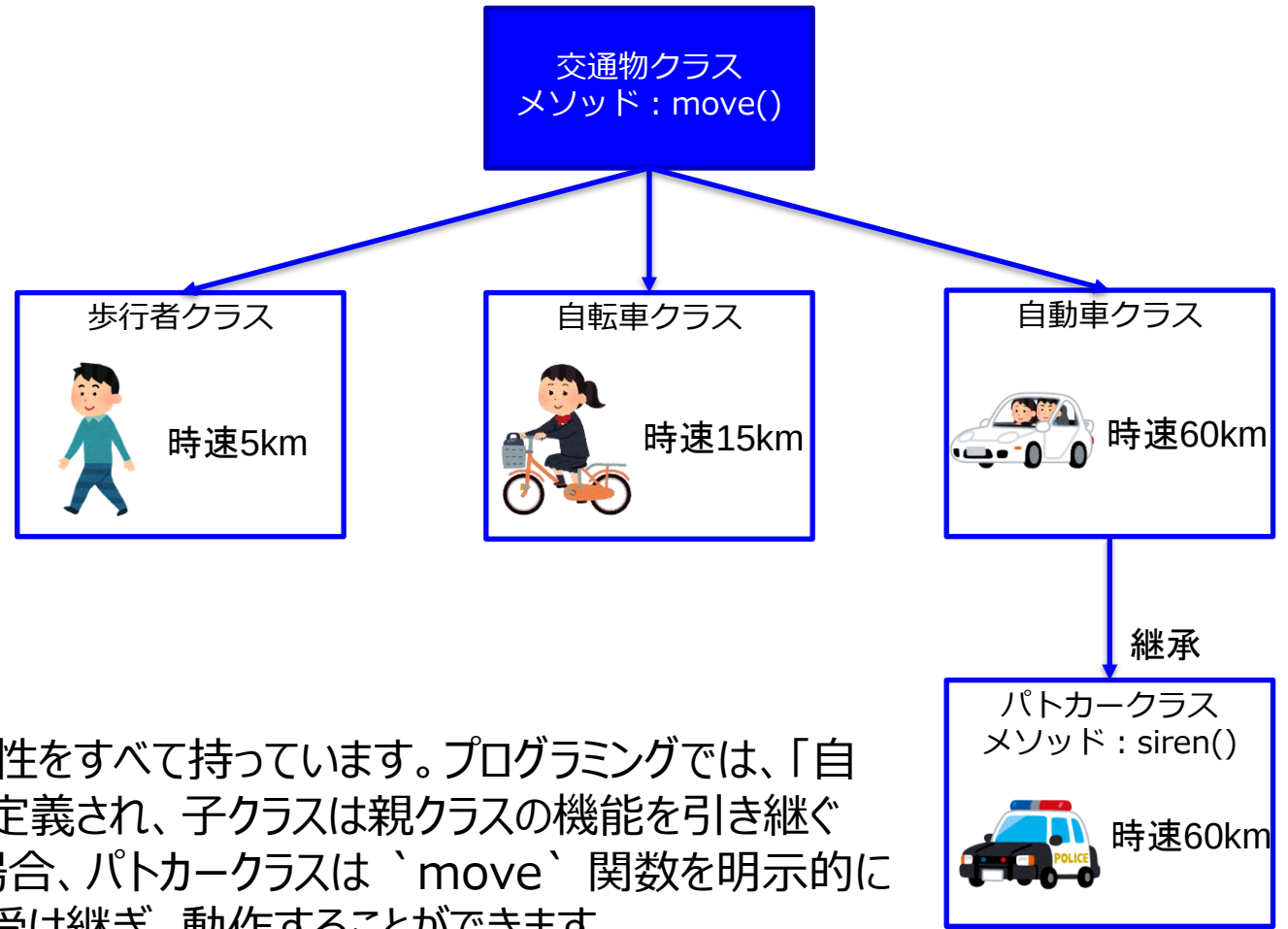
オブジェクト指向言語の一例（車を作る）

開発環境

プログラミング

● オブジェクト指向の基本概念

- ポリモーフィズム（Polymorphism）とは、関数やメソッドが、引数の数やデータ型の違いに応じた複数の実装を持ち、同じ関数名でも呼び出し時に適切な処理を実行できるプログラムの性質を指します。
- 関数は、どのように呼ばれるかによって異なる動作をしてほしい場面が多くあります。例えば、move()という関数があった場合、歩行者・自転車・自動車それぞれに適した移動の処理が必要になります。同じ関数名でも、呼び出し時の条件に応じて適切な処理を実行することができます。
- パトカーは自動車の一種であり、自動車の特性をすべて持っています。プログラミングでは、「自動車」が親クラス、「パトカー」が子クラスとして定義され、子クラスは親クラスの機能を引き継ぐ（継承する）という形で表現されます。この場合、パトカークラスは`move`関数を明示的に定義しなくても、自動車クラスからその機能を受け継ぎ、動作することができます。

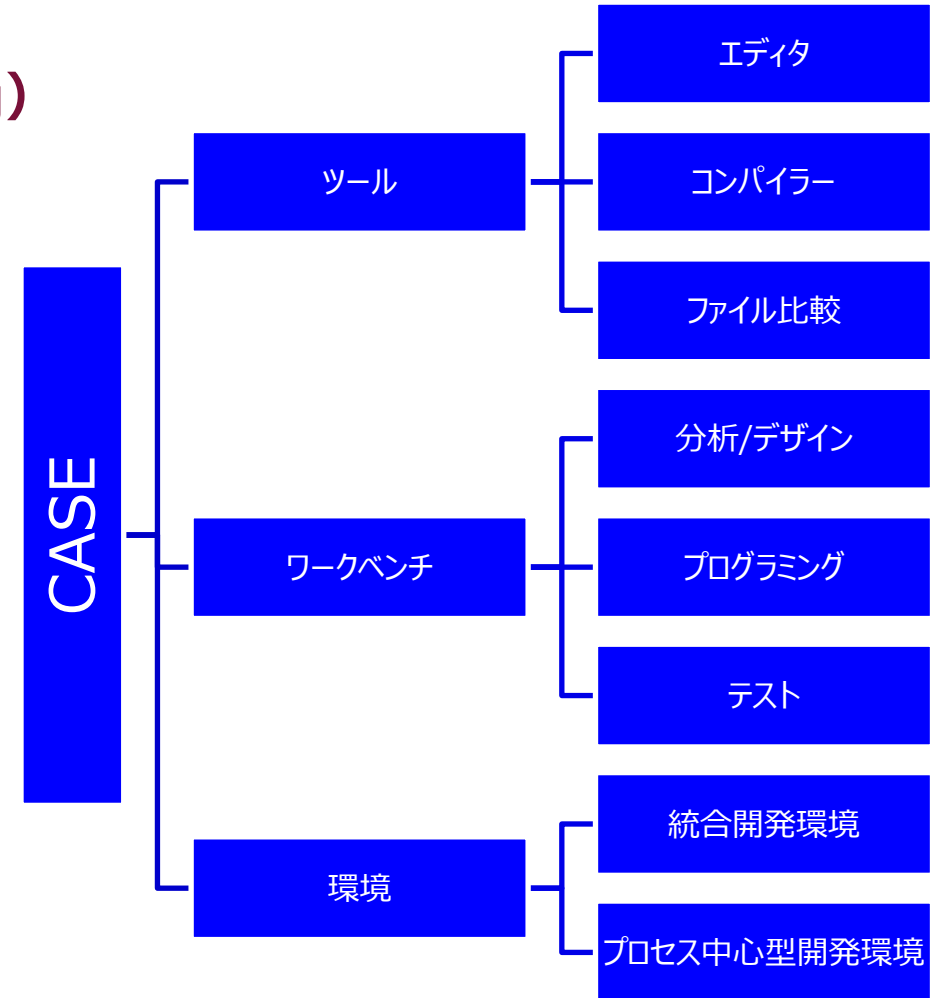


開発環境

コンピュータ支援ソフトウェア工学（CASE）

●CASE（Computer Aided Software Engineering）

- コーダーのソフトウェア開発のために、バグの検出の自動化やデバッグなどの開発に必要な機能の提供します。
- 発想としては、ハードウェア製品の設計に使用されるコンピュータ支援設計（CAD）ツールに似ています。
- CASEソフトウェアは、次の3つのカテゴリに分類されます。
- ツール（Tool）は、ソフトウェアライフサイクルの特定のタスクをサポートします。
- ワークベンチ（Workbench）は、ソフトウェアライフサイクルの特定の部分に焦点を合わせた2つ以上のツールを組み合わせたものです。
- 環境（Environment）は、2つ以上のツールまたはワークベンチを組み合わせて、ソフトウェアのライフサイクル全体をサポートします。





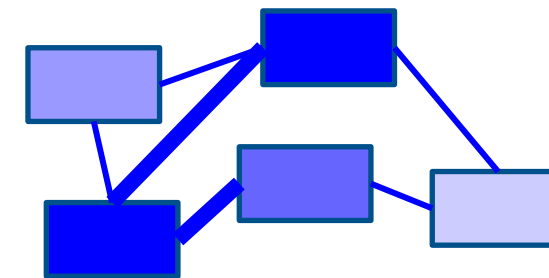
依存関係

結合度／凝縮度

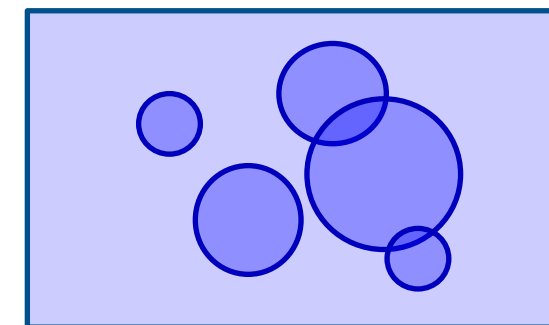
結合度 (Coupling)

- ソフトウェアモジュール間の相互依存の度合いで、2つのルーチンやモジュールがどれだけ密接に依存しているかを示す指標です。
- 結合度が低い（疎結合）ほど、モジュールが独立しており、変更が容易になります。

- ソフトウェアモジュール内の機能の相互依存の度合いで、モジュール内の要素がどの程度まとまっているかを示す指標です。
- 凝縮度が高いほど、モジュールの責務が明確になり、再利用しやすくなる。



結合度 (Coupling)



凝縮度 (Cohesion)

依存関係

商用オフザシェルフ（COTS）

● 商用オフザシェルフ

- 商用オフザシェルフ（COTS、Commercial Off-The-Shelf）とは、一般市場向けに販売されている既製の商用ソフトウェアやハードウェアのことです。
- COTS製品は、企業や個人がそのまま購入し、特別なカスタマイズなしで利用できる ように設計されています。
- COTS（Commercial Off The Shelf）またはカスタムビルドのソフトウェアを購入する場合、必要なセキュリティが確保されているかどうかを確認する必要があります。
- ベンダーが主張するセキュリティ事項は、それが証明されるまではマーケティング上の主張であると考えるべきです。つまり、製品を売りたいからちょっと良く言っているだけです。必要に応じて外部の調査機関を利用するとともに、十分な注意を払いましょう。

Haasの戦略

ハースF1チームは、アメリカを代表するフォーミュラ1レーシングチームです。F1チームの中では予算規模が小さいですが、合理的な戦略により、アメリカ発のチームとして、F1界に新しい風を吹き込みつつ、国際的な競争に挑戦しています。

エンジン供給や技術支援については、フェラーリとの提携関係があり、これにより最新技術の恩恵を受けています。しかし、ハースはエンジンや一部の技術でフェラーリに大きく依存しているため、フェラーリ側の技術的なトラブルや制限が直接影響を及ぼすリスクがあります。また、自社独自の技術開発における自律性が低いという点も挙げられます。



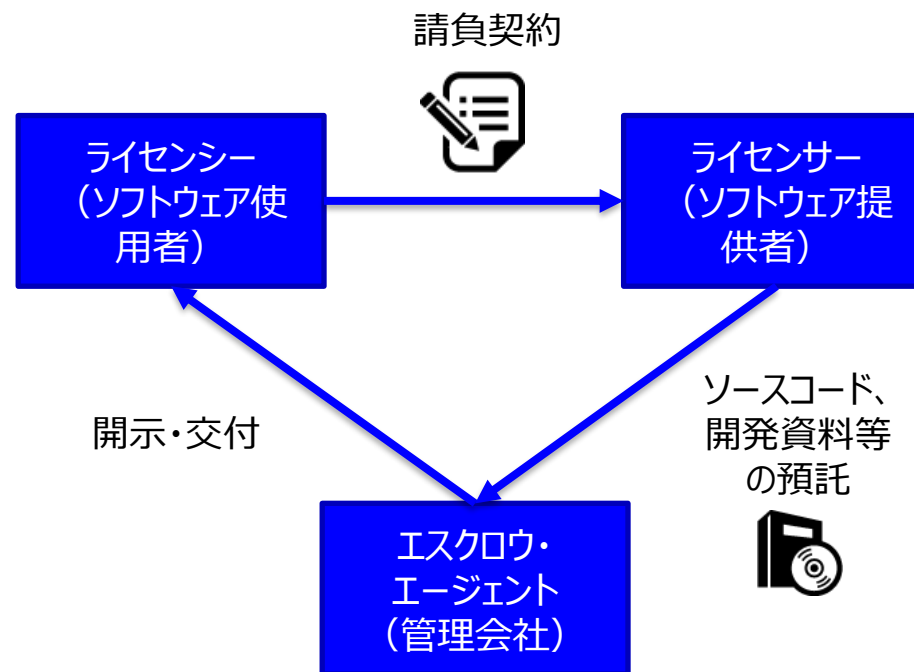
依存関係

商用オフザセルフ (COTS)

●ソフトウェア開発／運用のバックアップ

ソフトウェア・エスクロウ

- ソフトウェア・エスクロウとは、ソフトウェアのライセンスを販売している以外の組織に預けることです。
- 企業向けソフトウェアには、利用する権利であるライセンスの購入が必須です。しかし、企業が倒産してしまうとライセンスを提供することができなくなります。こうなると、次に困るのがライセンスを購入して、使っている企業です。そのため、ライセンスを第3組織（エスクローエージェント）に預け、運用を続ける方法があります。

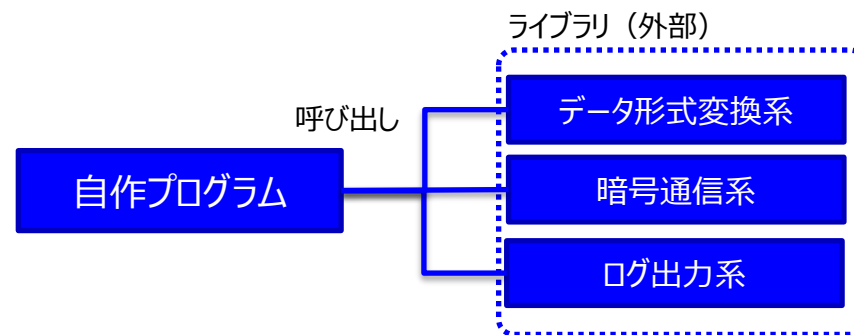


依存関係

オープンソース

●ソフトウェアライブラリ

- 開発者は、ソフトウェアライブラリ（使い回せるソフトウェア）を共有しています。
- ライブラリは機能を包括的に提供し、開発する上では欠かせません。もしもあなたのソフトウェアが他のアプリケーションとネットワーク通信したいときに、ネットワーク通信のためのロジックを理解し、かつセキュリティに問題がないものを一からプログラムするのは大変な作業です。
- OpenSSLといった、TLS通信を提供するライブラリを利用し、安全で迅速に実装することができます。ソフトウェアは広く使われることで多くの開発者からバグの共有が行われ、さらに素晴らしいソフトウェアになります。



プログラミングする際に、読み込んだライブラリ内のクラス、関数を呼び出すように記述する。

依存関係

オープンソース

● オープンソース

オープンソース (Open source)

- ソースコードを、無償で一般公開し、テスト、改善、修正できるものです。
- 単一の作成者や企業による開発体制である必要性はなく、有志のコミュニティによって開発されているため、プロプライエタリソフトウェアよりも安価で、長期にわたって利用できる傾向があります。
- 公開されているため、攻撃者がコードの欠陥を見つけることもできます。

クローズドソース (Closed source)

- ソフトウェアのバイナリをリリースしますが、ソースコードはリリースしないものになります。
- 営業秘密に基づいた健全なビジネスの慣行であると同時に、コードレベル処理を“隠す”ことによるセキュリティでもあります。

プロプライエタリソフトウェア (Proprietary software)

- 知的財産または特許によって保護されているソフトウェアです。
- クローズドソースと同じ意味ではありません。クローズドソースは、ソースコードを秘蔵にするだけであり、権利に関しては定義上触れていません。

依存関係

オープンソース

●ソフトウェアライセンス

GPL (GPL License、GNU General Public License)

- ユーザーがソフトウェアを実行、調査、共有、および変更する自由を保証するライセンスです。

BSD (Berkeley Software Distribution)

- カリフォルニア大学バークレー校が配布したUNIX系OSおよび関連ソフトウェア群です。対象となるソフトウェアの使用と再配布に最小限の制限を課しており、多くの商用あるいはフリーの派生ソフトウェアを生み出しました。

項目	GPLライセンス	BSDライセンス
商用利用	可能	可能
ソースコードの公開義務	必須	不要
変更後の再配布	GPLライセンス	任意のライセンス
例	Linux、GCC、MySQL	FreeBSD、PostgreSQL

依存関係

マネージドサービス

● マネージドサービス

- マネージドサービス（Managed Service）は、クラウドプロバイダーや専門のサービス提供者が運用・管理を行うITサービスです。例えば、AWSのRDS（データベース）、GCPのCloud Run、AzureのApp Serviceなどが代表例です。

項目	利点	欠点
運用負担	インフラ管理が不要	カスタマイズが制限
スケーラビリティ	自動スケール機能	予想以上のコスト増加
セキュリティ	セキュリティパッチが自動更新	データ管理がプロバイダー依存

依存関係

データベース操作

●データベースの正規化

- データベースの正規化とは、同一テーブル内のデータの重複をなくすために、適宜データを分割・整理することです。冗長データを削除し、データベースの整合性と可用性を向上させます。

注文番号	注文者	商品コード1	注文1	数量1	仕入コード1	農場名1	商品コード2	注文2	数量2	仕入コード2	農場名2
00001	A男	YASAI1	トマト	1	NOZYO 1	やさい農場	KONA1	小麦粉	3	NOZYO 1	やさい農場
00001	A男	YASAI2	キャベツ	2	NOZYO 2	こなこな農場	KONA2	薄力粉	4	NOZYO 2	こなこな農場

- 上の表では、各属性にデータが入ったちゃんとしたテーブルに見えますが、注文という属性と数量という属性が複数あったりと、非常に煩雑です。注文1と数量1という属性名でデータ同士が紐づいているのでしょうか？このようなデータの格納をしていると使い手のアプリケーションがデータベース構成をかなり意識することになります。
- 正規化のメリットは、重複を排除することでデータの不整合が起きるリスクを減らせることです。

依存関係

データベース操作

●データベースの正規化

第1正規形

- 第1正規形では、同一行内での繰り返しをなくします。

注文テーブル

注文番号	注文者	注文 1	数量 1	注文 2	数量 2
00001	A男	トマト	1	キャベツ	2
00002	A子	小麦粉	3	薄力粉	4



注文テーブル

注文番号	注文者	注文	数量
00001	A男	トマト	1
00001	A男	キャベツ	2
00002	A子	小麦粉	3
00002	A子	薄力粉	4

第2正規形

- 第2正規形では、ある属性を識別するための項目である主キーとすることでテーブルとします。
- 属性の部分関係からテーブルを分離します。

注文テーブル

注文番号	注文者	商品コード	注文	数量
00001	A男	YASAI1	トマト	1
00001	A男	YASAI2	キャベツ	2
00002	A子	KONA1	小麦粉	3
00002	A子	KONA2	薄力粉	4



注文テーブル

注文番号	注文者	商品コード
00001	A男	YASAI1
00001	A男	YASAI2
00002	A子	KANA1
00002	A子	KANA2

商品テーブル

商品コード	注文	数量
YASAI1	トマト	1
YASAI2	キャベツ	2
KONA1	小麦粉	3
KONA2	薄力粉	4

依存関係

データベース操作

●データベースの正規化

第3正規形

- 第3正規形では、テーブル内の一部の属性がキーでない項目によって決定されることです。
- 属性の部分関係からテーブルをさらに分離します。

注文テーブル

注文番号	注文者	商品コード	注文	数量	仕入れコード	農場名
00001	A男	YASAI1	トマト	1	NOZYU 1	やさい農場
00001	A男	YASAI2	キャベツ	2	NOZYU 2	こなこな農場
00002	A子	KONA1	小麦粉	3	NOZYU 1	やさい農場
00002	A子	KONA2	薄力粉	4	NOZYU 2	こなこな農場



注文テーブル

注文番号	注文者	商品コード
00001	A男	YASAI1
00001	A男	YASAI2
00002	A子	KANA1
00002	A子	KANA2

商品テーブル

商品コード	商品名	仕入コード
YASAI 1	トマト	NOZYU 1
YASAI 2	キャベツ	NOZYU 2
KONA 1	小麦粉	NOZYU 1
KONA 2	薄力粉	NOZYU 2

仕入先テーブル

仕入コード番号 (FK)	農場名
NOUZYU 1	やさい農場
NOUZYU 2	こなこな農場

依存関係

データベース操作

●データベース構造

データベーススキーマ

- データベーステーブルの属性と値の構造です。
- 名前には文字列しか入れられないようにデータには型や名前と電話番号はセットで保管すべきといった関係性を定義します。

データディクショナリ

- データベーステーブルといったメタデータの説明するものになります。
- データベースビュー情報、許可されたデータベース管理者に関する情報、ユーザーアカウントの名前と権限、監査情報、データベーススキーマなどがあります。

データベースビュー

- 既存のテーブルのデータを基に作成される仮想的なテーブルです。
- DBにアクセスする側から見ると、一つのテーブルとして見ることができます。
- 参照元のテーブルの値が変更すればビューテーブルの値も変更されます。

商品テーブル

商品コード	商品名	仕入コード番号
YASAI 1	トマト	NOZYO 1
YASAI 2	キャベツ	NOZYO 2
KONA 1	小麦粉	NOZYO 1
KONA 2	薄力粉	NOZYO 2



「YASAI*」を抽出して
テーブルを作る。

野菜商品テーブル

商品コード	商品名	仕入コード番号
YASAI 1	トマト	NOZYO 1
YASAI 2	キャベツ	NOZYO 2

データベースビュー

依存関係

データベース操作

● データベース操作

データ定義言語 (DDL、Data Definition Language)

- データベースの構造を定義するコマンドです。

データ操作言語 (DML、Data Manipulation Language)

- データベースのデータの選択、挿入、削除、および更新を行うコマンドです。

データ制御言語 (DCL、Data Control Language)

- アクセス権の制御やデータベースの状態管理を行うコマンドです。
- DBMSでは、管理したいDBのためのDBMS用のアカウントを管理しています。各アカウントに対して権限を与え、DBの操作を許しています。
- DBを利用するときには、アプリケーションはDBMSに指定のアカウントでログインし、DBを操作することになります。

データ定義言語の例

- 作成を行う (CREATE)
- 削除する (DROP)
- 変更を加える (ALTER)
- データを全削除する (TRUNCATE)

データ操作言語の例

- データの検索を行う (SELECT)
- データを追加する (INSERT)
- データを更新する (UPDATE)
- データを削除する (DELETE)

データ制御言語の例

- 権限を与える (GRANT)
- 権限を消す (REVOKE)
- トランザクションを確定する (COMMIT)
- トランザクションを破棄する (ROLLBACK)

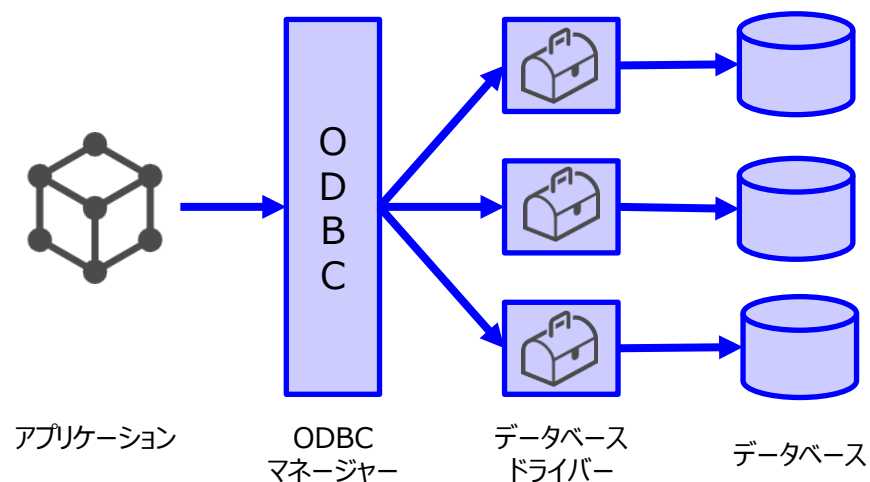
依存関係

データベース操作

● データベース操作

ODBC (Open Database Connectivity)

- データベース管理システム (DBMS) にアクセスするプログラムに対して、DBに依存せず共通のインターフェースを与えるための仕様です。
- 世の中には様々なDBがありますが、DBに依存してどのデータベースごとに接続設定が異なると非常に厄介です。
- ODBCは、共通のDB情報設定を与え、アプリケーションに対して共通のインターフェースを提供できます。



ODBCによりアプリケーションはDBの接続仕様が共通化される



運用と保守

運用と保守

DevSecOps

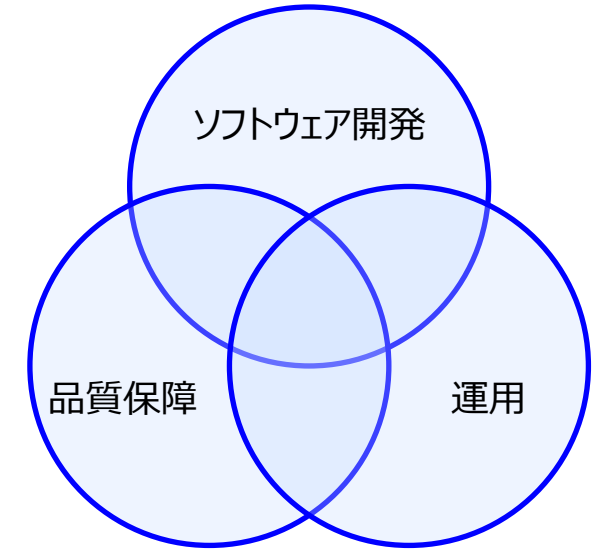
● 開発と運用

DevOps

- DevOps (Development + Operations) とは、開発 (Development) と運用 (Operations) を統合し、ソフトウェアの開発からリリース、運用までをスムーズに進めるための考え方やプラクティス です。
- 例 : CI/CD, IaC, モニタリング

DevSecOps

- DevSecOps (Development + Security + Operations) とは、DevOpsのプロセスに「セキュリティ」を統合し、開発・運用の早い段階からセキュリティ対策を組み込むアプローチ です。
- 例 : SAST, DAST, コンテナセキュリティ



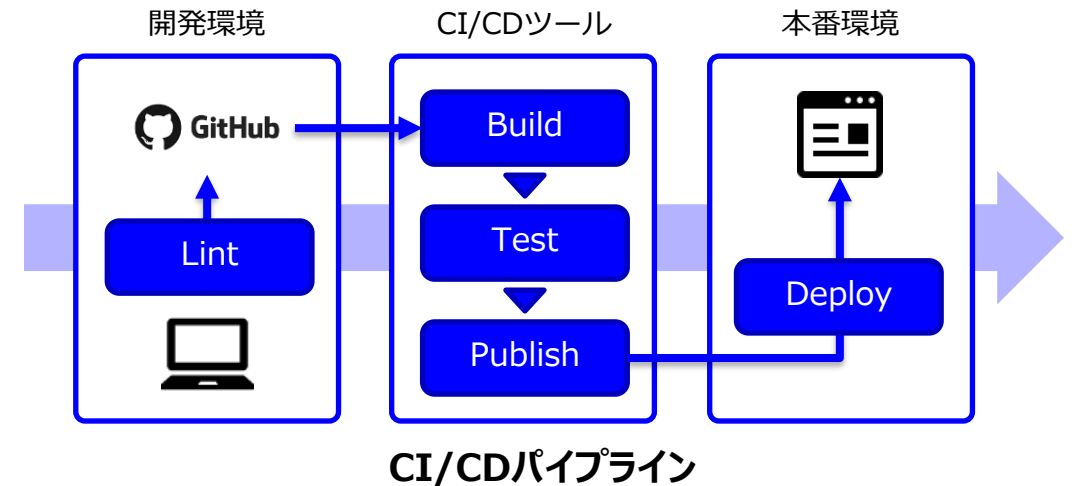
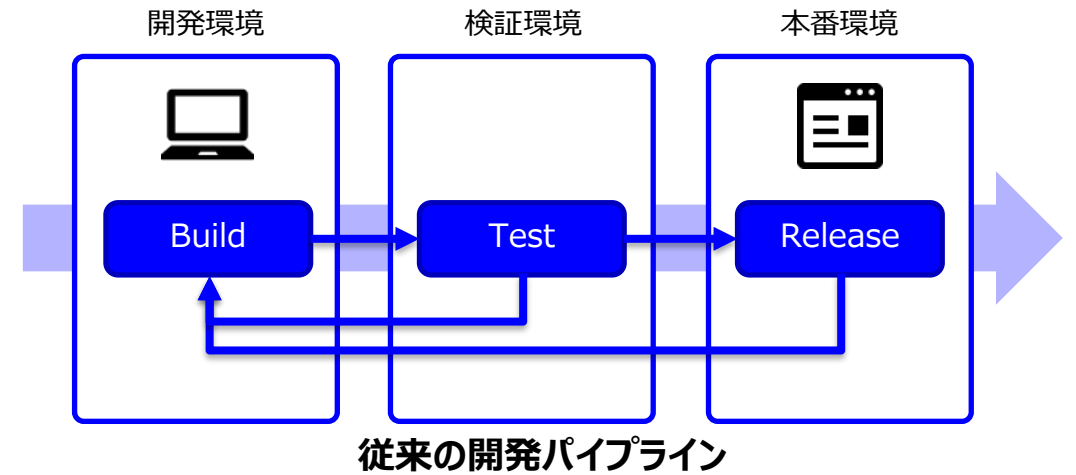
DevOpsモデル

運用と保守

継続的インテグレーション／継続的デリバリー（CI/CD）

● CI/CD（継続的インテグレーション／継続的デリバリー）

- CI/CD（Continuous Integration / Continuous Delivery）とは、ソフトウェア開発の自動化を目的とした手法で、コードのリント、ビルド、テスト、パブリッシュ、デプロイのプロセスを効率化する開発モデルです。
- リントとは、コードの静的解析を行い、潜在的なエラーやコーディング規約違反を検出するプロセスです。
- ビルドとは、ソースコードを実行可能なプログラムに変換するプロセスです。
- テストとは、ソフトウェアが期待通りに動作するかを確認するプロセスです。
- パブリッシュとは、ビルドされた成果物を外部に公開したり配布したりする準備プロセスです。
- デプロイとは、ソフトウェアをサーバー、クラウド、またはユーザーのデバイスなどの運用環境に配置し、実際に利用可能な状態にするプロセスです。



運用と保守

ソフトウェアコンポーネント分析（SCA）

●ソフトウェア部品表（SBOM）

- SBOM（Software Bill of Materials）とは、ソフトウェアを構成するすべてのコンポーネント（部品）を一覧にしたりリストです。これは、ソフトウェアの透明性を高め、セキュリティリスクやライセンス管理を適切に行うために使用されます。

SBOMの目的と利点

- ソフトウェアの構成を可視化（どのライブラリやモジュールが使われているか明確にする）
- 脆弱性管理を強化（SBOMを使って脆弱なコンポーネントを特定し、迅速に対応可能）
- ライセンス管理の最適化（GPLやBSDなどのオープンソースライセンス違反を防ぐ）
- サプライチェーン攻撃のリスク低減（信頼できないコンポーネントの混入を防ぐ）

要素	説明
コンポーネント名	使用されているソフトウェア部品（ライブラリモジュール）の名前
バージョン	各コンポーネントのバージョン情報
ライセンス情報	コンポーネントが適用するオープンソースライセンスの種別
発行元	ライブラリやソフトウェアの提供元
依存関係	どのコンポーネントが他のコンポーネントと依存しているか
ハッシュ値	ソフトウェア部品の整合性を検証するためのチェックサム

運用と保守

アプリケーションセキュリティテスト（AST）

●アプリケーションセキュリティテスト

- **SAST（Static Application Security Testing）** とは、ソースコードやバイナリを解析して脆弱性を検出するテスト手法です。コードを実行せずに脆弱性をチェックできるのが特徴です。
- **DAST（Dynamic Application Security Testing）** とは、実行中のアプリケーションに対して外部から疑似攻撃を行い、脆弱性を検出するテスト手法です。
- **IAST（Interactive Application Security Testing）** とは、アプリケーションの実行中に内部のデータフローを解析し、リアルタイムで脆弱性を検出するテスト手法です。

項目	SAST	DAST	IAST
解析対象	ソースコード	実行中のアプリケーション	実行中のアプリケーション + 内部データフロー
適用フェーズ	開発	テスト	テスト・運用
脆弱性の特定	高い	低い	高い
誤検知	少ない	少ない	少ない
代表的なツール	SonarQube, Checkmarx	OWASP ZAP, Burp Suite	Contrast Security, Veracode IAST

運用と保守

ソフトウェアの公開

●ソフトウェアの配布

- **フリーウェア (Free software)** とは、すべての機能を無料で利用できるソフトウェアです。
- **シェアウェア (Shareware)** とは、基本的に有料だが、一部または一定期間無料で試せるソフトウェアです。例えば、ソフトウェアをテストするためにトライアル期間の30日間は無料で使えるが、以降使用を継続するには料金を支払う必要があるなどの料金体系をしている場合にはシェアウェアに該当します。
- **機能限定ソフト (Crippleware)** とは、無料で利用できるが、重要な機能が制限されているソフトウェアです。シェアウェアとの違いは、試用期間がないが無料版の機能が限定される点にあります。

●ソフトウェア利用許諾契約 (EULA、End-User License Agreements)

- ソフトウェアの生産者や開発者から購入者又は利用者間で行われる契約です。
- 一般的には、ユーザーがソフトウェアのインストールした後、利用を開始する前にソフトウェアの利用規約に「同意する」をクリックする電子的なフォーム画面で成立します。



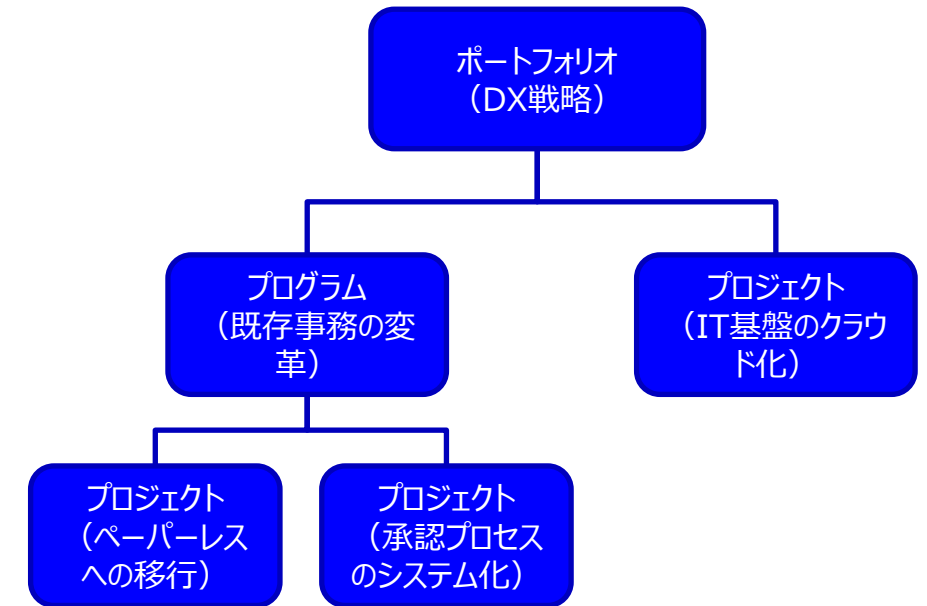
開発プロセスモデル

開発プロセスモデル

プロジェクト管理

●プロジェクト

- プロジェクトは一時的な取り組みであり、独自の製品、サービス、または結果の作成に焦点を当てた有限の時間での活動です。
- ポートフォリオは、企業の戦略的目標を達成するためのプロジェクトとプログラムの集合です。
- プログラムは、プロジェクトの集合であり、開始と終了の時期を持って完了としません。継続的な改善などが求められるものは、プログラムの目標を達成するために個々のプロジェクトによって目的を達成しようとしています。



ポートフォリオ、プログラム、プロジェクトの関係性

開発プロセスモデル

プロジェクト管理

● 体制

プロジェクトマネージャー/プロジェクトリーダー

- プロジェクトマネージャーとは、プロジェクトの全体管理を担い、計画・進捗・リスク・コスト・品質を統括する責任者です。
- プロジェクトリーダーとは、チームの現場リーダーとして、メンバーの技術的・業務的なサポートを行う役割です。

統合製品チーム（IPT、Integrated Product Team）

- 統合製品チームとは、異なる専門分野のメンバーが協力して、製品開発の各プロセスを統合的に進めるチームのことです。
- 通常、エンジニア、デザイナー、品質管理、マーケティング、プロジェクトマネージャーなど、複数の部門から構成され、各分野の専門知識を活かしながら、製品開発のスピードと品質を向上させることを目的としています。

「君、とりあえず見積もってくれたまえ。」

要件はないが見積るときがある。ここまで矛盾した事はあるのだろうか。残念ながらこのような言葉はIT業界にはびこっています。ITシステム構築の工数を見積もるには、過去の実績から基にする、WBSを使い細かく積み上げる、楽観・悲観・可能性の3パターンを考慮する、システム機能数をベースに計算、ソースコードの規模で算出、プロジェクトチームで見積もる方法に分けられます。お金が絡むと緊張する人もいますが、手法を知っていると楽になれます。

ただ、あとから要件が変更されたり、変更されたうえでの詳細な根拠を求められたりするので、やはり納得はできない。

開発プロセスモデル

プロジェクト管理

●プロジェクト管理

ガントチャート (Gantt chart)

- プロジェクトの各段階を作業単位まで階層構造で示す表であり、縦軸に作業内容・担当者・開始日・終了日・作業間の関連情報を記載し、横軸に時間を記載します。
- ガントチャートは、いつからいつまでのどの作業をしなければならないのかを確認でき、あなたのプロジェクトの進捗管理の助けになります。どのプロジェクトでも作成することになるでしょう。

WBS (Work Breakdown Structure)

- プロジェクトを作業の最小単位まで階層的に細分化した構成図になります。

タスク	1	2	3	4	5	6	7	8	9	10
機能設計										
アプリ開発										
インフラ設定										
動作テスト										

ガントチャート

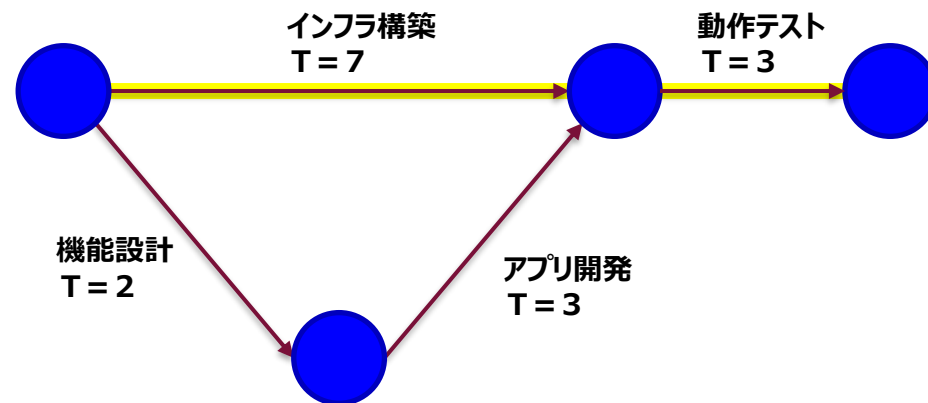
開発プロセスモデル

プロジェクト管理

●プロジェクト管理

PERT (Program Evaluation and Review Technique)

- 各工程の先行関係を図示することで全体の所要期間を見積もったり、大きな遅れにつながるような工程を見極めるための手法です。
- PERTでは、各状態の先行関係を矢印で図示します。先行関係の結果、所要時間が最大となる経路はどこで、どれか遅れると全体の所要時間も増えるのかクリティカルパスを探します。



アプリ開発は終わるが、インフラ構築が終わらないとテスト環境にデプロイできない

開発プロセスモデル

ウォーターフォール

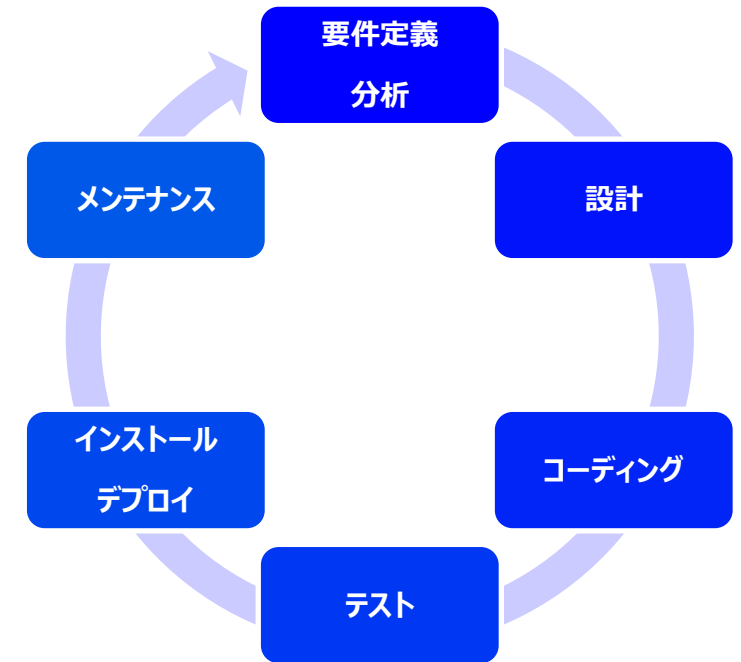
● ライフサイクルモデル

SDLC (Software Development Life Cycle)

- ソフトウェアの要件定義、基本設計、詳細設計、開発・導入、試験、運用・保守までの工程を指す言葉です。SDLCは方法論ではありません。あくまでソフトウェアの検討から保守するまでを定義する概念になります。
- 各フェーズを明確に定義することで、各フェーズでの確認する作成物を認識合わせすることができ、顧客の期待を満たすシステムを開発することに繋がります。

各フェーズにおけるセキュリティも意識できる

- 設計では標準的な仕様に則っているのか、どれだけ標準的な構成であるのかなどを確認し、コーディングのフェーズでは開発者自身がセキュアなコーディングテクニックを熟知しているかが重要です。



ソフトウェア開発ライフサイクルの各フェーズ

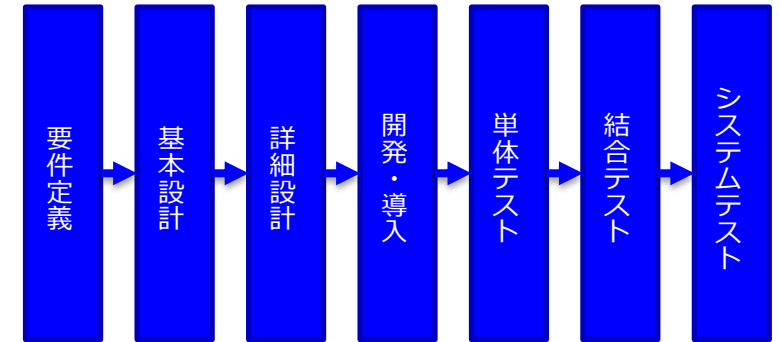
開発プロセスモデル

ウォーターフォール

● ライフサイクルモデル

ウォーターフォールモデル

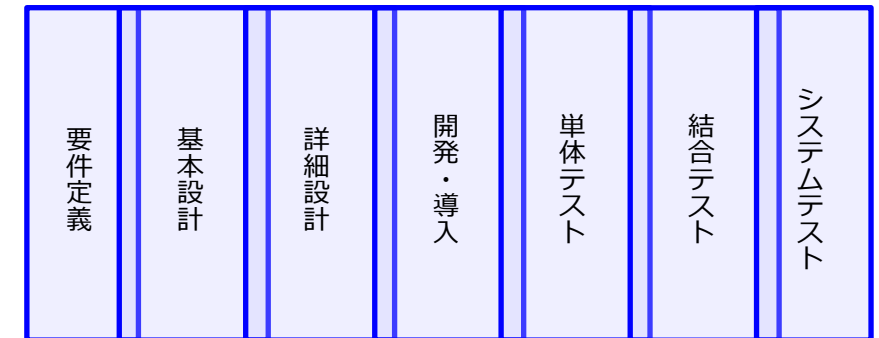
- 各工程で成果物を確認しながら進めていきます。ウォーターフォールモデルでは後戻りということが許されません。
- 多くの認識合わせを実施しなければならない大規模な開発では、シンプルな意思決定が求められるため、よく用いられる手法です。



ウォーターフォールモデル

SASHIMIモデル

- 各フェーズの終点と始点の並行稼働を許したシステム開発プロセスのモデルです。
- SASHIMIモデルは、フェーズ間のコミュニケーションを即座にできる中規模プロジェクトに適しています。システム開発は前フェーズの成果物を前提に後フェーズの作業が決まるため、前フェーズでの修正点を即時で後フェーズの作業者に反映する意思伝達が必要になります。



SASHIMIモデル

開発プロセスモデル

アジャイル

●アジャイルソフトウェア開発

- 適応計画、進化的開発、早期提供、継続的改善に重きを置き、トライアルアンドエラーな開発手法です。
- アジャイルは従来のプロセスをモデル化する考え方とは異なり、主義や主張をチーム全体で共有し、あらゆる状況にも対応する試みです。

アジャイルソフトウェア開発のマニフェスト

- プロセスやツールよりも、個人と相互作用
- 包括的なドキュメントよりも動作するソフトウェア
- 契約交渉よりも、顧客との協調
- 計画の順守よりも、変化への対応

マニフェストの12の原則

- 顧客満足を最優先し、価値のあるソフトウェアを早く継続的に提供すること
- 開発が遅れ、後期に要求の変更が来たとしても歓迎すること
- 動作するソフトウェアは頻繁に共有すること（数か月ではなく数週間程度が目安）
- ビジネスマンと開発者は、プロジェクトを通して日々密接に協力し合うこと
- 信頼されるべき意欲のある個人を中心にチームが構成されること
- 情報を伝えるもっとも効率的で効果的な方法是对面での会話です
- 動作するソフトウェアこそが進捗の主要な尺度です
- 持続可能な開発を促進し、一定のペースを継続的に維持すること
- 卓越した技術と優れた設計への継続的な関心をもつこと
- シンプルさ（無駄のない開発を最大化する事）が本質です
- 最高のアーキテクチャ、要件、および設計は、自己組織化チームから生まれる
- 定期的な振り返りにより、チームはより効果的になる方法を検討し、それに応じて調整する

開発プロセスモデル

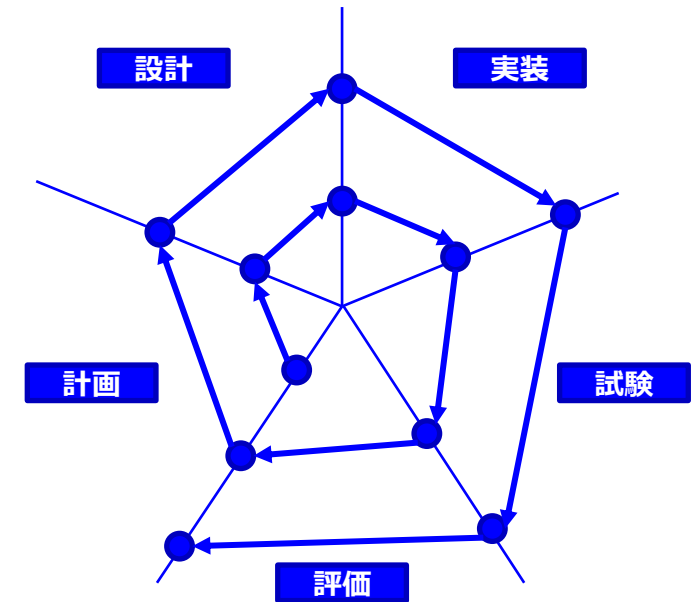
アジャイル

●スパイラルモデル

- 計画・分析・設計・実装・試験・評価から成る一連のプロセスを一つのプロジェクト内で何度も繰り返し、次第に完成度を高めていく方式です。
- ソフトウェアプロジェクトでは、これらのフェーズを繰り返します。

●RAD (Rapid Application Development)

- 少人数のチームでプロトタイプの開発を行い、評価・改良することで徐々に完成度を高めていく方式です。
- プロトタイプは最終開発物であるソフトウェアの開発の設計仕様を加えていくと同時に、設計仕様の確認として使用されます。
- 顧客の要求が抽象的であるソフトウェアの開発は、テキストでの認識合わせが難しいため、実際に動いているモノがある方が非常に適しています。



スパイラルモデル

開発プロセスモデル

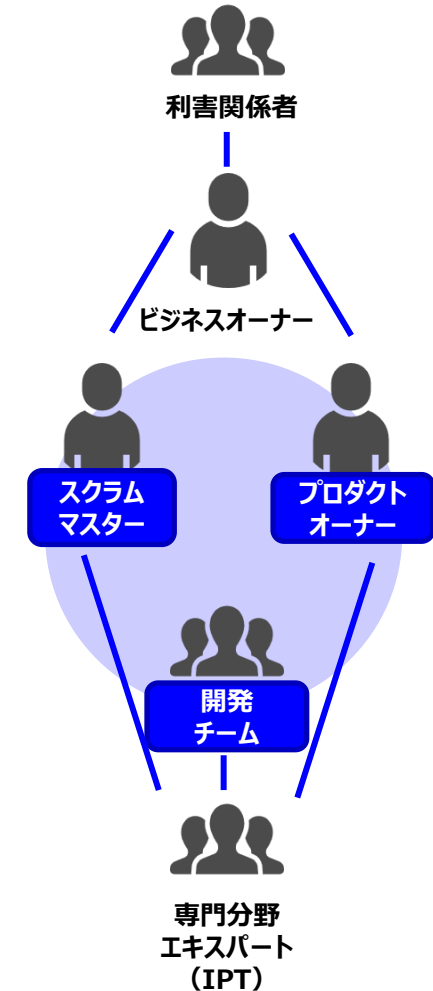
アジャイル

●スクラム

- 約10人のチーム向けに設計されている、スプリントと呼ばれる2週間の開発サイクルと毎日の短いスタンドアップミーティングで構成されるアジャイル開発手法の一つです。

スクラムフレームワークの3コアの役割

- プロダクトオーナーとは、製品の利害関係者、顧客の声を代表し、チームがビジネスに価値を提供することを保証する責任があります。
- 開発チームとは、各スプリントの最後に製品を提供する責任があります。チームは、実際の作業（分析、設計、開発、テスト、テクニカルコミュニケーション、ドキュメントなど）を行う3～9人で構成されます。
- スクラムマスターとは、製品の目標と成果物を提供するチームの能力に対する障害を取り除くことを促進し、説明責任を負います。



開発プロセスモデル

アジャイル

●エクストリームプログラミング（XP、Extreme Programming）

- ソフトウェアの品質と、変化する顧客の要件への応答力を向上させることを目的としたアジャイル開発手法の一つです。

XP用途のピックアップ

- ペアプログラミング（二人一組でプログラミングし、一人はコーディング、一人はレビューを主に行う事）で開発し、てくことで、広範なコードレビューの実行すること。
- 実装の前にテストを作成し、実装はテストをパスすることを前提に開発を行っていきます。（テスト駆動開発）
- コードの単純さと明快さを持つこと。（リファクタリング、You Aren't Going to Need It）
- フラットな管理構造を持ち、現在の全体状況を把握できるようにチームに共有すること。
- 時間が経過し、問題がよりよく理解されるにつれて、顧客の要件が変化することを想定すること。

開発プロセスモデル

成熟度モデル

●ソフトウェアCMM (SW-Capability Maturity Model)

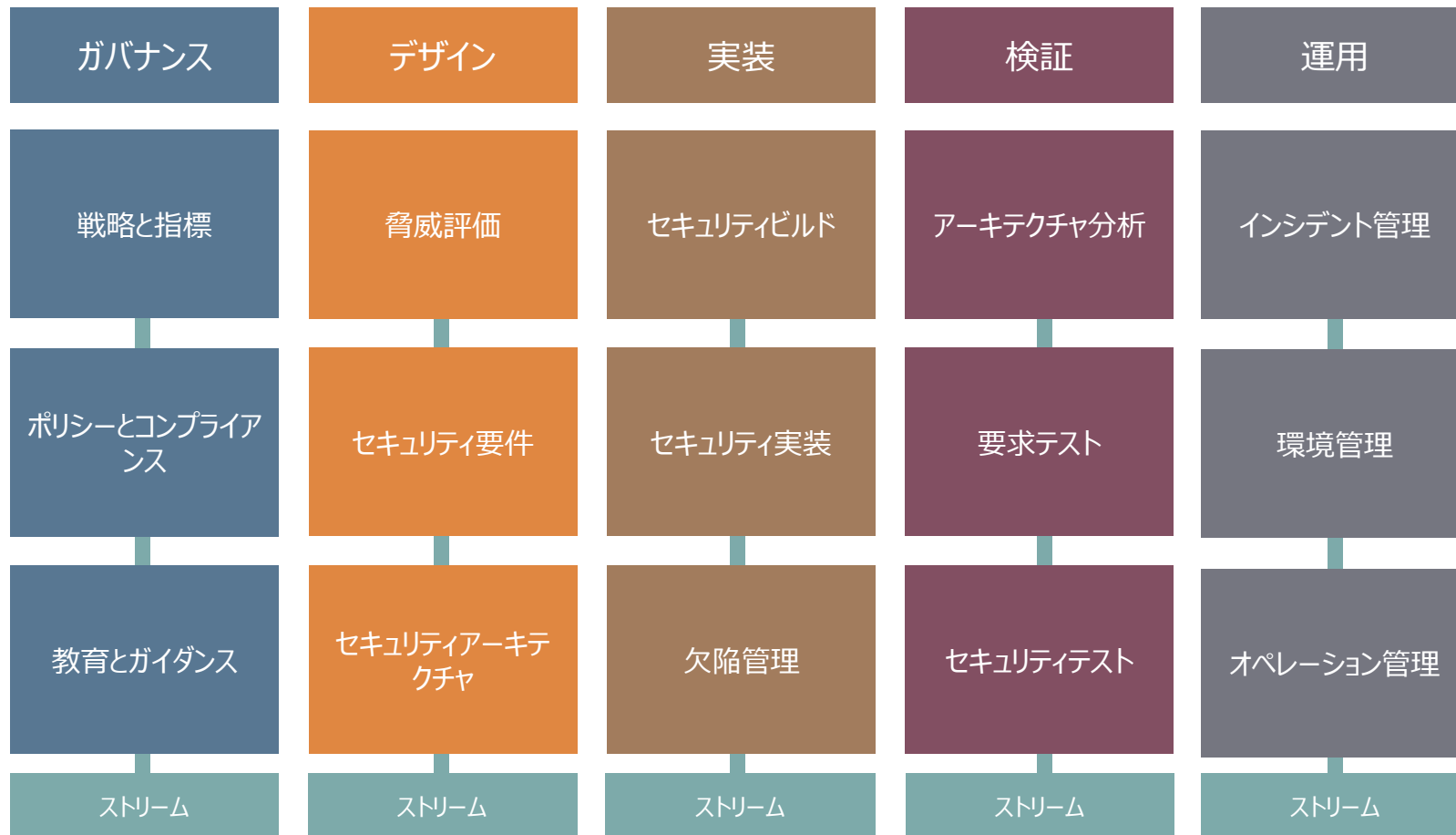
レベル	説明
レベル 1 : 初期	- このレベルでは、通常文書化されておらず、動的に変化している状態です。 - ユーザーやイベントによって、その場しのぎで、制御されていない、反応的な方法で動かされる傾向があります。 - そのため、プロセスは混沌とした不安定な状態にあります。
レベル2 : 反復可能	- このレベルでは、成熟度ではいくつかのプロセスが反復可能であり、一貫した結果が得られるでしょう。 - プロセスの規律が厳格になることはありませんが、存在する場合には既存のプロセスが維持されることを保証するのに役立ちます。
レベル3 : 定義された	このレベルでは、文書化された一連の標準プロセスが確立されており、時間をかけてある程度改善されています。
レベル4 : 管理された	- このレベルでは、目標達成のためにプロセスの評価が行われています。 - プロセスの利用者は、複数の様々な条件でプロセスを経験し、能力を実証することができます。
レベル 5 : 最適化	このレベルでは、漸進的かつ革新的な技術的改善により、プロセス性能を継続的に改善すること状態です。

開発プロセスモデル

成熟度モデル

● ソフトウェア保証成熟度モデル (Software Assurance Maturity Model)

- SAMMの目的は、セキュアな開発ライフサイクルを分析し、改善するための効果的で測定可能な方法を提供することです。
- SAMMは、技術やプロセスにとらわれず、ソフトウェアのライフサイクル全体をサポートします。
- SAMMは、すべての組織に通用する単一のやり方は存在しないため、進化的でリスク主導型の性質を持つように作られています。



開発プロセスモデル

アーキテクチャ決定記録

●アーキテクチャ決定記録（Architectural Decision Record、ADR）

- アーキテクチャ決定記録（Architectural Decision Record、ADR）とは、システム開発における重要なアーキテクチャの意思決定を文書化し、記録するためのドキュメントです。
- 開発チームがなぜ特定のアーキテクチャ設計を採用したのかを明確にし、将来的な変更や意思決定の参考にするために使用されます。

項目	説明
Title	決定内容の要約（例：「データベースの選定」）
Status	提案（Proposed） / 承認済み（Accepted） / 却下（Rejected）など
Context	なぜこの意思決定が必要なのか、背景や課題を説明
Decision	最終的に採用した技術やアーキテクチャの選択肢
Consequences	期待されるメリットやデメリット、影響範囲
Alternatives Considered	検討したが採用しなかった他の選択肢



セキュアコーディング

セキュアコーディング

OWASP

●OWASP (Open Web Application Security Project)

- OWASP (Open Web Application Security Project) とは、ソフトウェアやWebアプリケーションのセキュリティ向上を専門とし、研究やガイドラインの作成、脆弱性診断ツールの開発、イベント開催など実施するコミュニティです。
- OWASPの基本理念は、すべての資料と情報が無料で、誰でもWebサイトから簡単にアクセスできることです。OWASPは、ツール、ビデオ、フォーラム、プロジェクトからイベントまで、あらゆるものを提供します。

OWASP Top10

- Webアプリケーションのセキュリティに関する最も重大な10のリスクをランキングとして公開し、その修正のガイダンスを提供しています。
- OWASP Top10は、2～3年ごとに世界のセキュリティ状況に応じて更新されます。
- ソフトウェア開発ライフサイクル (SDLC) に組み込むことは、セキュア開発を行う上では素晴らしい判断です。

位	項目
1	アクセス制御の不備
2	暗号化の失敗
3	インジェクション
4	安全が確認されない不安な設計
5	セキュリティの設定ミス
6	脆弱で古くなったコンポーネント
7	識別と認証の失敗
8	ソフトウェアとデータの整合性の不具合
9	セキュリティログとモニタリングの失敗
10	サーバーサイド・リクエスト・フォージェリ

OWASP Top10

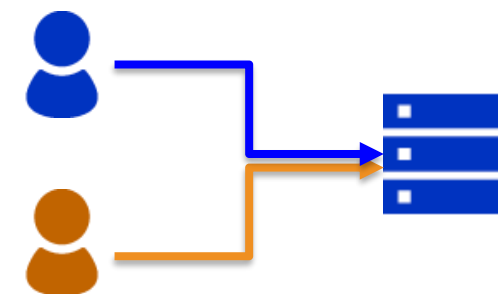
セキュアコーディング

OWASP

●OWASP Top10

A01:2021 アクセス制御の不備

- アクセス制御の不備があると、攻撃者が他のユーザーになりすまし、システムへ不正にアクセスできる可能性があります。これは、データの機密性を適切に認識・管理していない場合にも発生します。
- あるアプリケーションで適切に制御されていても、別のアプリケーションでは不十分な場合があり、一貫性のあるアクセス制御メカニズムが必要です。
- 特に、管理者用のユーザーアカウントやインターフェースは狙われやすく、不適切な制御があるとサイトのユーザー、データ、コンテンツが危険にさらされるため、厳格な管理が求められます。



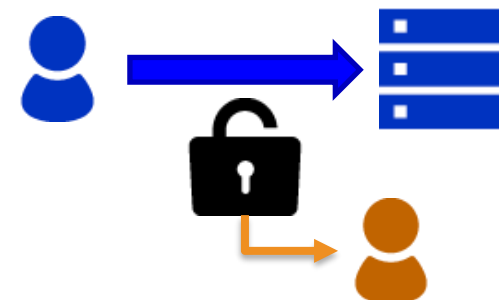
セキュアコーディング

OWASP

●OWASP Top10

A02:2021 暗号化の失敗

- 暗号技術の不備は、機密データの漏洩やシステムの侵害につながる重大なリスクです。
- 例えば、サイトが HTTPS ではなく HTTP の場合、ユーザーがブラウザから送信するメールアドレスやパスワードなどのデータが暗号化されず、盗聴の危険性が高まります。
- バックアップデータや通信中のデータが暗号化されていないと、漏洩時に深刻な影響を及ぼす可能性があります。さらに、古い非推奨の暗号化方式を使用していると、たとえ暗号化していても攻撃者が簡単な計算ツールで解読できるため、安全とは言えません。
- データの流出を監視していないと、万が一情報が漏洩した際に、ユーザーへ適切な対応や警告を行うことができません。したがって、最新の暗号技術の導入と継続的な監視が不可欠です。



セキュアコーディング

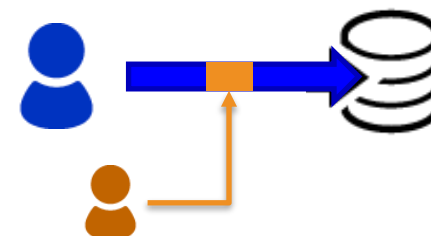
OWASP

●OWASP Top10

A03:2021 インジェクション

- インジェクション攻撃とは、攻撃者が通常の入力インターフェースを利用し、意図的に異常なデータを送信することで、アプリケーションに想定外の操作を実行させる攻撃手法です。
- 特に、SQLインジェクション（SQLi）やLDAPインジェクションなどが代表的な例として挙げられます。

- データ型の適正なチェック（例：文字列・数値・日付の明確な制限）
- 入力の長さ制限（異常に長いデータの送信を防ぐ）
- 特殊文字の無害化やエスケープ処理（', ", --, <script> などの不正な入力を防ぐ）
- 適切なUIの利用（例：名前は文字のみ入力可、電話番号は数値のみ入力可、国や州の選択にはドロップダウンリストを使用することで不正入力を防止）



セキュアコーディング

OWASP

●OWASP Top10

A04:2021 安全が確認されない不安定な設計

- セキュリティ設計や試験を開発の早い段階で実施するためには、脅威モデリングや安全な設計に対する理解を深めることが重要です。
- セキュリティ設計に欠陥が見つかった場合、単なる修正ではなく、根本的な設計の誤りを疑うべきです。設計レベルでの問題が解決されない限り、同様の脆弱性が繰り返し発生する可能性があります。
- セキュリティ上の問題が発見されても、それが適切に報告されなかったり、対応が遅れたり、その場しのぎの修正にとどまると、侵害に対して適切な対策が講じられたとは言えません。
- 今後、同様の攻撃に対しても効果的な防御策を取るために、ソフトウェアやアプリケーションにパッチを適用し、不要なポートを閉じるなどの対策を確実に実施することが不可欠です。



セキュアコーディング

OWASP

●OWASP Top10

A05:2021 セキュリティの設定ミス

- セキュリティの設定ミスとは、システムの設計や構成における誤りや適切でない設定が原因で発生する脆弱性を指します。
- 例えば、デフォルトのアクセス権限や設定が削除されていない場合、管理者が意図せず脆弱な状態を放置してしまう可能性があります。また、製品のデフォルトのユーザー名やパスワードを変更せずに使用していると、同じ製品の仕様を知っている攻撃者が容易にシステムへ侵入できるというリスクがあります。
- OS、ウェブサーバー、データベース（DBMS）、アプリケーションなどに適切なパッチが適用されず、最新の状態に保たれていない場合も、セキュリティの設定ミスに該当します。定期的な更新と適切な設定管理が不可欠です。



セキュアコーディング

OWASP

●OWASP Top10

A06:2021 古くなった脆弱なコンポーネント

- 既知の脆弱性を含むコンポーネントを使用してアプリケーションを開発・実行することで発生する問題です。
- 開発者は、使い慣れていることを理由に、すでに非推奨となったコードや、安全でないと判明しているライブラリ・モジュールを採用してしまうことがあります。しかし、これらのコンポーネントには既知の脆弱性が含まれているため、攻撃者に狙われるリスクが高まります。
- 最新のライブラリやフレームワークを活用し、定期的にコンポーネントを更新することで、古くなった脆弱なソフトウェアを使用するリスクを最小限に抑えることが重要です。



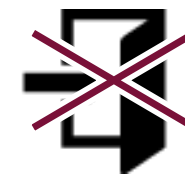
セキュアコーディング

OWASP

●OWASP Top10

A07:2021 識別と認証の失敗

- 認証やセッション管理の機能が適切に実装されていない場合、攻撃者はパスワードやセッション情報を不正に取得し、システムへ侵入するリスクがあります。
- 特に、安全でない認証方式や不適切なセッション管理（例：セッションの有効期限が長すぎる、適切に無効化されないなど）があると、攻撃者によるアカウントの乗っ取りが容易になります。
- 強力なパスワードポリシー、多要素認証（MFA）の導入、セッションの適切な管理を行うことで、このリスクを軽減できます。



セキュアコーディング

OWASP

●OWASP Top10

A08:2021 ソフトウェアとデータの整合性による不具合

- ソフトウェアの更新や重要なデータの管理は、CI/CDパイプラインの不整合によって問題が発生しやすい領域です。
- CI/CDパイプラインは、開発者向けの継続的インテグレーションとデリバリーを自動化するプロセスですが、これにより依存モジュールや追加機能がリアルタイムで更新されることで、バージョンの競合や整合性の問題が生じる可能性があります。
- これを防ぐためには、ソフトウェアの署名検証、整合性チェック、依存関係の管理を適切に行い、安全なリリースプロセスを確保することが重要です。



セキュアコーディング

OWASP

●OWASP Top10

A09:2021 セキュリティログとモニタリングの失敗

- ロギング（記録）とモニタリング（監視）は、Webサイトの運用において不可欠なプロセスです。
- アプリケーションの処理を適切にログに記録し、定期的に監視しなければ、セキュリティ侵害の検知が遅れたり、適切な対応が困難になる可能性があります。
- 効果的なセキュリティ対策として、リアルタイム監視、異常なアクセスの検知、ログの適切な保管と分析 を行うことが重要です。



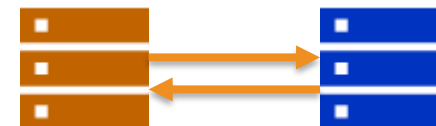
セキュアコーディング

OWASP

●OWASP Top10

A10:2021 サーバーサイド・リクエスト・フォージェリ

- サーバーサイド・リクエスト・フォージェリ（SSRF） は、攻撃者が他のWebサイトを経由して、自社のシステムに不正なリクエストを送信させる攻撃手法 です。
- この脆弱性を悪用されると、セッションIDやトークン、クッキーに保存されたパスワードやユーザー名が漏洩するリスク があります。
- 特に、ログオフされていないセッションや、他サイトからのリダイレクトによるアクセスを許可している場合、攻撃者が不正なリクエストを送り込むことが可能になります。
- 対策として、送信データがあるWebページには固有のセッション・トークンを埋め込み、不正なトークン値のリクエストを拒否する ことで、SSRF攻撃を防ぐことができます。

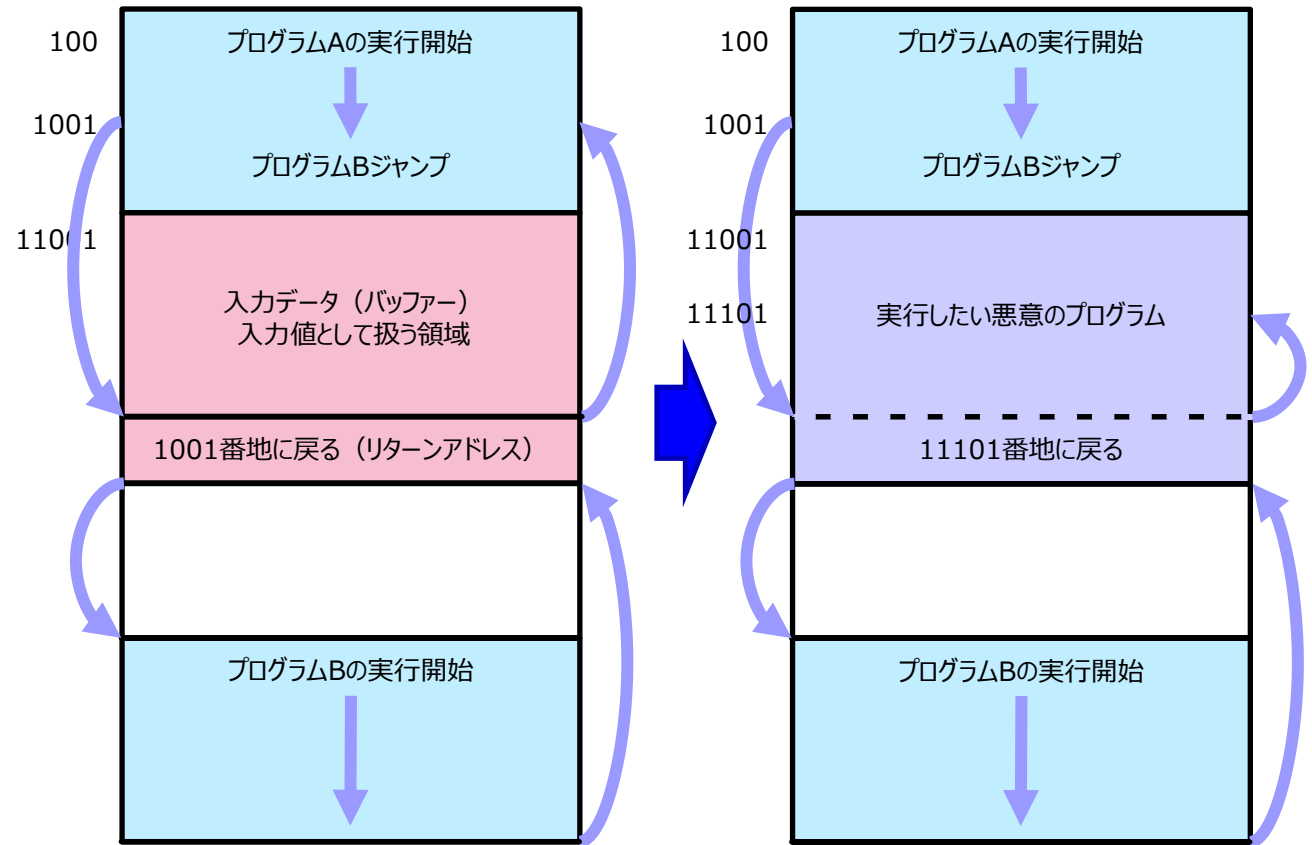


セキュアコーディング

標的になるコード

● バッファオーバーフロー（バッファオーバーラン）

- バッファとは、データを保持するために用意されたメモリ領域のことです。プログラムのあるセクションから別のセクションへ、あるいはプログラム間でデータを移動する際に使用されます。
- バッファオーバーフローは、不正な入力によって引き起こされます。すべての入力が一定のサイズ以下であると仮定し、バッファがそのサイズになるように作成されている場合、異常なトランザクションがより大きなデータを生成すると、バッファの終わりを超えて書き込むことができます。
- 隣接するデータや実行コードが上書きされてしまうと、メモリアクセスエラー、不正な結果、クラッシュなど、プログラムの動作が不安定になる可能性があります。



通常は入力値とリターンアドレスは
分かれている

指定よりも大きな入力値によって、
リターンアドレスを上書きし、入力値とし
て記録したプログラムを実行する

セキュアコーディング

標的になるコード

● 競合

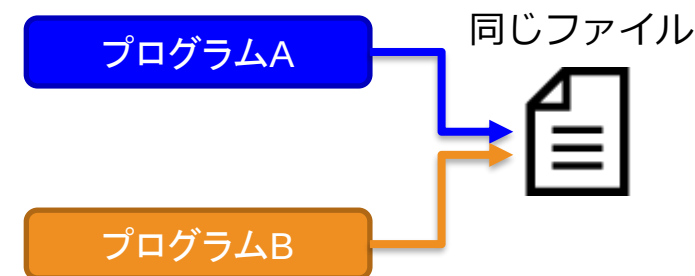
レース状態（レースハザード）

- レース状態（レースハザード / Race Condition）とは、複数のスレッドやプロセスが同時にリソースへアクセスすることで、意図しない競合や不整合が発生する問題です。この問題は、並行処理（マルチスレッド / マルチプロセス）環境において、処理の順序が制御されていない場合に発生します。

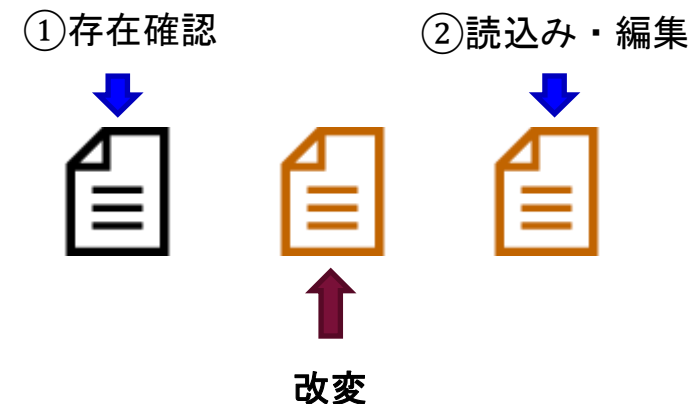
TOC/TOU（time of check to time of use）

- ある条件をチェックしてから、そのチェック結果を使用するまでの間にシステムが変更されることで発生するソフトウェアのバグです。

レース状態（レースハザード）



TOC/TOU



セキュアコーディング

標的になるコード

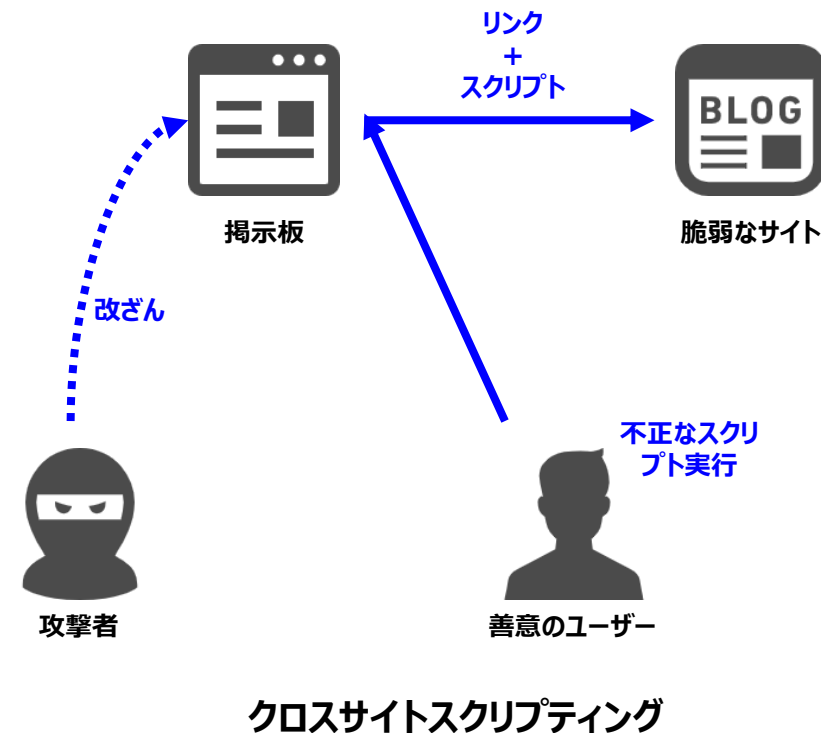
●スクリプト言語

Cross Site Scripting

- 掲示板などから脆弱なサイトに誘導をかけつつ、脆弱なサイトでのスクリプトを実行文も送付することで、ユーザーに本来存在しないはずのスクリプトを実行させる攻撃です。

Cross Site Request Forgeries

- 悪意のあるスクリプトを埋め込んだWebページに利用者を誘導し、不正なリクエストを利用者に送らせる攻撃です。
- 特定の投稿サイトやEコマースにリクエストを送付するようにスクリプトをWebページに埋め込んでおきます。このWebページを利用者が踏んでしまうと、勝手に投稿したり勝手に購買したりしてしまいます。
- 確認画面を入れるなどして一回で処理を実行できないにしたり、ページごとにランダム値を埋め込み入力値とともに称号をかけるなどの対応策があります。



セキュアコーディング

標的になるコード

●SQLインジェクション

- SQLインジェクション（SQL Injection）とは、アプリケーションのデータベースに対して不正なSQL文を挿入し、データを盗み出したり改ざんしたりする攻撃手法です。
- 攻撃者は、通常の入力フィールドに悪意のあるSQLコードを埋め込むことで、アプリケーションのセキュリティを突破し、データベースの情報を操作できます。
- プレースホルダを使って指定するようアプリケーション開発を行い、その部分はいくまで値として処理され、SQL命令に関わるような特殊文字は無効化される処理が必要です。

accountsテーブルから検索文字列を検索するSQL文

```
SELECT * FROM accounts WHERE id='検索文字列';
```



```
a'; TRUNCATE TABLE accounts;
```

実行されるSQL文

```
SELECT * FROM accounts WHERE id='a';  
TRUNCATE TABLE accounts; '
```

accountsテーブルから'a'を検索します
accountsテーブルのデータを削除

セキュアコーディング

標的になるコード

●ソフトウェア対策

バリデーションチェック

- 入力値には、文字種や文字列長をチェックしてからプログラムの入力値をして受け入れることです。
- バッファオーバーフローやSQLインジェクションなどの意図しない入力値を受け入れないようにします。

認証とセッション管理

- 認証が十分な強度を持っていても、セッションが推測可能なものであると意味を成しません。
- ログイン状態を示すセッション値は悪意のあるユーザーが推測できないように、十分な長さで乱数性を持った文字列を発行しなければなりません。

エラーハンドリング

- プログラム時にエラーが発生した際には、その後の調査を行う必要があります。デバッグモードを用意し、開発者の原因究明のための実行モードを用意することが望まれます。
- エラー発生した際に意図しないような実行されないように、ブルースクリーンなどの異常状態の報告を行う必要があります。

セキュアコーディング

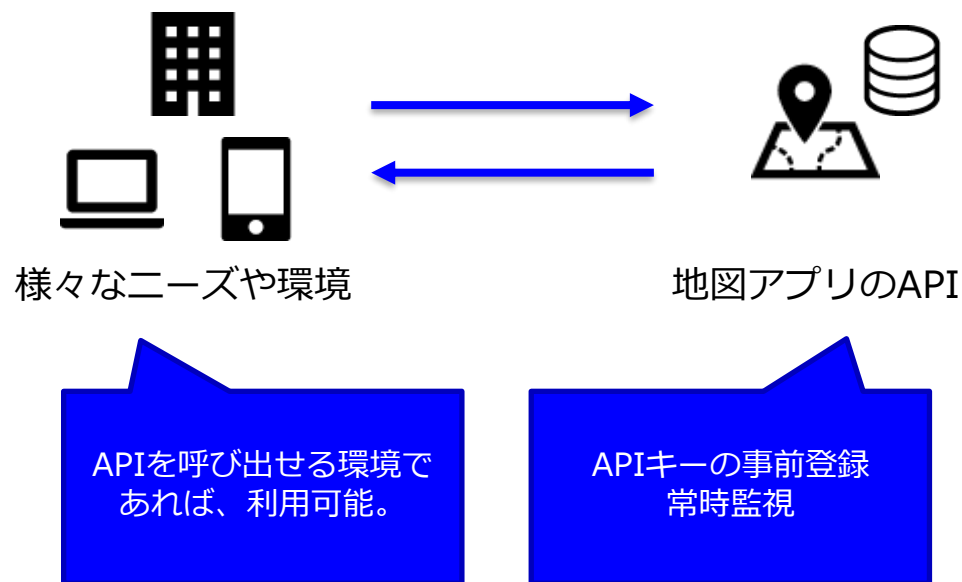
APIセキュリティ

●API（アプリケーションプログラミングインターフェイス）

- アプリケーションが別のアプリケーション、オペレーティングシステム、データベース、ネットワークなどと通信できるようにします。
- OWASPには、APIコントロールを含むEnterprise Security APIToolkitsプロジェクトがあります。

APIゲートウェイ（API Gateway）

- APIゲートウェイ（API Gateway）とは、クライアント（ユーザーやアプリ）とバックエンドのサービス（API）を仲介し、トラフィックを管理する役割を持つサーバーまたはサービスです。
- APIゲートウェイを通じて、セキュリティ・認証・ルーティング・負荷分散・監視などの機能を一元的に管理 することができます。



セキュアコーディング

APIセキュリティ

●オブジェクトブローカー

ORB (Object Request Broker)

- ORBは、異なるコンピュータ上のプログラムをネットワーク経由で呼び出せるようにするミドルウェア です。これにより、アプリケーションがどのコンピュータ上で実行されているかを意識せずに通信を行える「ロケーションの透明性」を提供します。
- 代表的なオブジェクトブローカーとして、.NET Remoting、COM、DCOM、CORBA などがあります。

COM (Component Object Model)

- COMは、プログラミング言語に依存せずにオブジェクトを実装し、異なる環境でも利用できる技術 です。これにより、異なる言語で作成されたコンポーネント同士を統合し、再利用しやすくなります。

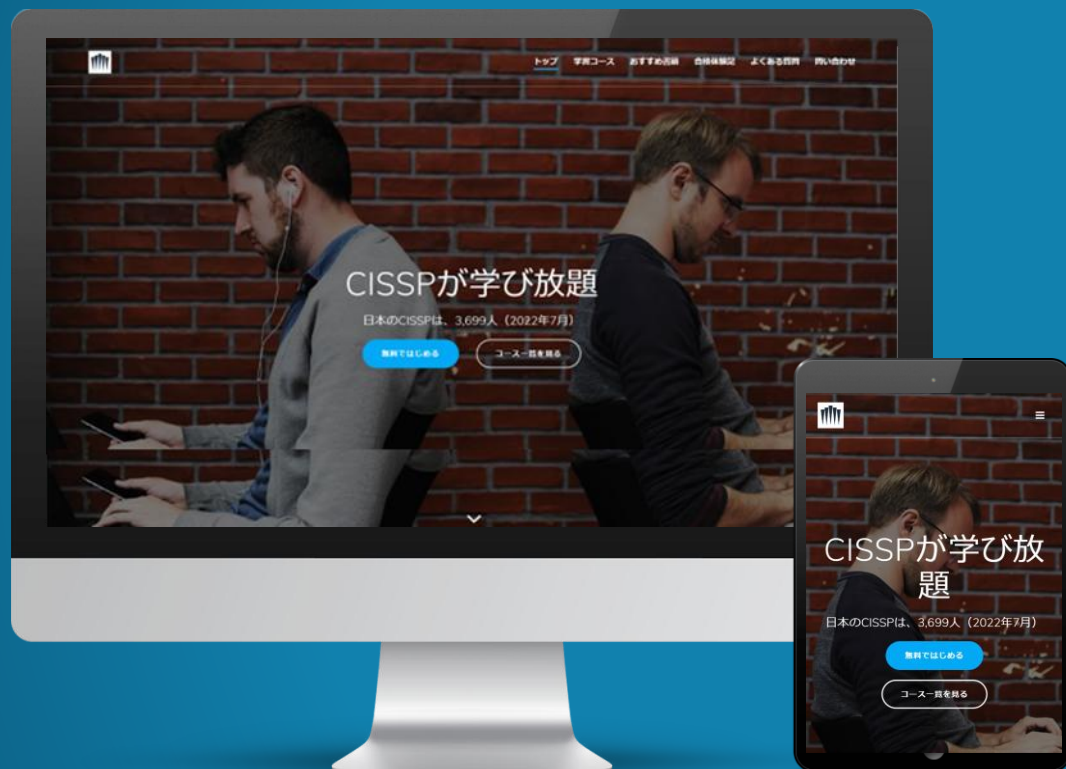
DCOM (Distributed Component Object Model)

- DCOMは、ネットワークを介してCOMコンポーネント同士が通信し、データのやり取りや処理のリクエストを行う技術 です。
- ソフトウェア開発では、多くの機能を一から開発するのではなく、既存のソフトウェアコンポーネントを再利用することで、効率的に開発を進めることが重要 です。現在では、COMやDCOMはMicrosoft .NETに徐々に置き換えられつつあります。

CORBA (Common Object Request Broker Architecture)

- CORBAは、OMG (Object Management Group) によって定義された、異なるプラットフォーム上のシステム間の通信を容易にする技術 です。これにより、異なるオペレーティングシステム、プログラミング言語、ハードウェア環境の間でもスムーズなデータのやり取りが可能 になります。

PIEDPINの紹介



なぜセキュリティを学ぶのか

セキュリティの最も大きな敵は、人はあらゆるものに慣れ、そのリスクを考えなくなる習性にあります。セキュリティの学びを通して得られる貴重なものは、あらゆる事象に対して好奇心を持つことであり、それは目まぐるしい現代を生きるのに大いに役立つでしょう。

- EASY DESIGN
- SUPER EFFICIENT
- EVERYWHERE

PIEDPINを選ぶ理由



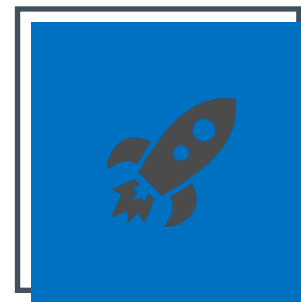
**EASY
DESIGN**

日本語で学べる



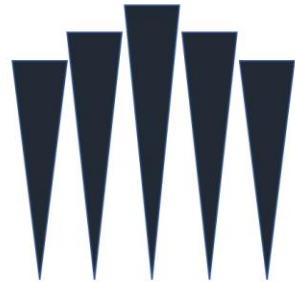
**SUPER
EFFICIENT**

わかりやすい説明



EVERYWHERE

どこからでも学べる環境



THANK YOU

CISSP for BEGINNER